

Homework 07 - Java Exception Handling

Topics Covered

- What is an exception
- Error vs Exception
- Compile-time error vs runtime exception
- `try`
- `catch`
- Multiple `catch`
- Catch block order
- `finally`
- Checked exception
- Unchecked exception
- Common runtime exceptions
- `throw`
- `throws`
- `throw` vs `throws`
- Custom exception
- `getMessage()`
- `printStackTrace()`
- Exception propagation
- Try-catch-finally with `return`
- Backend-style exception handling

Instructions

1. Write every program in a separate Java file.
2. First write the expected output, then run the program.
3. For every exception, write whether it is checked or unchecked.
4. For every error-fixing question, write the reason before correcting the code.
5. Do not use only `catch (Exception e)` everywhere. Use specific exception classes where possible.
6. After every coding question, write 2-3 lines explaining the exception flow.

Part A: Short Answer Questions

1. What is an exception?
2. Does exception occur at compile time or runtime?
3. What is the difference between compile-time error and runtime exception?
4. What is the difference between Error and Exception?
5. Why do we need exception handling?
6. What is the use of `try` block?
7. What is the use of `catch` block?

8. Can `try` be written without `catch` ?
9. Can one `try` block have multiple `catch` blocks?
10. How many catch blocks execute for one exception?
11. Why should child exception catch block come before parent exception catch block?
12. What is the use of `finally` block?
13. Does `finally` run when exception does not occur?
14. Does `finally` mean exception is handled?
15. What is a checked exception?
16. What is an unchecked exception?
17. Give three examples of checked exceptions.
18. Give five examples of unchecked exceptions.
19. What is the use of `throw` keyword?
20. What is the use of `throws` keyword?
21. What is the difference between `throw` and `throws` ?
22. What is a custom exception?
23. Why do we create custom exceptions?
24. What does `getMessage()` return?
25. What does `printStackTrace()` print?
26. What is exception propagation?
27. What happens when `finally` has a `return` statement?
28. Why should we avoid return inside `finally` ?
29. Why should we not silently catch exceptions?
30. In backend applications, why should we show user-friendly error messages?

Part B: Find The Output

Question 1

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
            System.out.println(result);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero");  
        }  
  
        System.out.println("Program completed");  
    }  
}
```

Question 2

```

public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Line 1");
            System.out.println(10 / 0);
            System.out.println("Line 2");
        } catch (ArithmeticException e) {
            System.out.println("Exception handled");
        }

        System.out.println("Line 3");
    }
}

```

Question 3

```

public class Main {
    public static void main(String[] args) {
        try {
            int[] numbers = {10, 20, 30};
            System.out.println(numbers[5]);
            System.out.println(10 / 0);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid index");
        } catch (ArithmeticException e) {
            System.out.println("Invalid division");
        }
    }
}

```

Question 4

```

public class Main {
    public static void main(String[] args) {
        try {
            String name = null;
            System.out.println(name.length());
        } catch (Exception e) {
            System.out.println("Something went wrong");
        } finally {
            System.out.println("Finally block");
        }
    }
}

```

Question 5

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            String value = "abc";  
            int number = Integer.parseInt(value);  
            System.out.println(number);  
        } catch (NumberFormatException e) {  
            System.out.println("Invalid number format");  
        }  
    }  
}
```

Question 6

```
public class Main {  
    static int test() {  
        try {  
            System.out.println("Inside try");  
            return 10;  
        } catch (Exception e) {  
            System.out.println("Inside catch");  
            return 20;  
        } finally {  
            System.out.println("Inside finally");  
        }  
    }  
  
    public static void main(String[] args) {  
        int result = test();  
        System.out.println("Result: " + result);  
    }  
}
```

Question 7

```

public class Main {
    static int test() {
        try {
            System.out.println("Inside try");
            int result = 10 / 0;
            return 10;
        } catch (Exception e) {
            System.out.println("Inside catch");
            return 20;
        } finally {
            System.out.println("Inside finally");
        }
    }

    public static void main(String[] args) {
        int result = test();
        System.out.println("Result: " + result);
    }
}

```

Question 8

```

public class Main {
    static int test() {
        try {
            return 10;
        } catch (Exception e) {
            return 20;
        } finally {
            return 30;
        }
    }

    public static void main(String[] args) {
        System.out.println(test());
    }
}

```

Question 9

```

class Calculator {
    void divide() {
        int result = 10 / 0;
        System.out.println(result);
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calculator = new Calculator();

        try {
            calculator.divide();
        } catch (ArithmeticException e) {
            System.out.println("Handled in main");
        }
    }
}

```

Question 10

```

public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Try started");
        } finally {
            System.out.println("Finally executed");
        }
    }
}

```

Part C: Identify The Exception

For each code, write:

1. Exception name
2. Checked or unchecked
3. Reason
4. Correct handling

Question 1

```
int result = 100 / 0;
```

Question 2

```
int[] marks = {80, 90, 70};
System.out.println(marks[10]);
```

Question 3

```
String name = null;
System.out.println(name.length());
```

Question 4

```
String value = "ten";
int number = Integer.parseInt(value);
```

Question 5

```
String city = "Delhi";
System.out.println(city.charAt(20));
```

Question 6

```
int[] numbers = new int[-5];
```

Question 7

```
Object value = "Java";
Integer number = (Integer) value;
```

Question 8

```
FileReader reader = new FileReader("student.txt");
```

Part D: Find And Fix The Error

Question 1

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
        }
    }
}
```

Write:

1. What is wrong?
2. Why is `try` alone not allowed here?
3. Correct the code.

Question 2

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
        } catch (Exception e) {
            System.out.println("General exception");
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic exception");
        }
    }
}
```

Write:

1. What is the error?
2. Why is the second catch unreachable?
3. Correct the catch order.

Question 3

```
import java.io.FileReader;

public class Main {
    public static void main(String[] args) {
        FileReader reader = new FileReader("data.txt");
    }
}
```

Write:

1. What is the exception?
2. Is it checked or unchecked?
3. Correct using try-catch.

Question 4

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
        } catch (Exception e) {
        }
    }
}
```

Write:

1. Why is this bad practice?
2. What problem can happen in real project?
3. Rewrite with meaningful message.

Question 5

```
class VoterService {
    void checkAge(int age) {
        if (age < 18) {
            new IllegalArgumentException("Age must be 18 or above");
        }
    }
}
```

Write:

1. What is missing?
2. Why will exception not be thrown?
3. Correct using `throw`.

Question 6

```
class FileService {
    void readFile() {
        FileReader reader = new FileReader("data.txt");
    }
}
```

Write:

1. What import is needed?
2. What exception must be handled or declared?
3. Correct using `throws`.

Part E: Coding Practice

Program 1: Division Calculator

Create a program that divides two numbers.

Requirements:

- If second number is zero, handle `ArithmeticException`.
- Print meaningful message.
- Print `"Program completed"` at the end.

Program 2: Array Index Handler

Create an integer array of 5 values.

Try to print an index entered by user.

Requirements:

- Handle `ArrayIndexOutOfBoundsException`.
- Handle `InputMismatchException`.
- Close `Scanner` in `finally`.

Program 3: String To Number Converter

Create a program that converts a String to int.

Test with:

```
"123"  
"abc"  
"10a"
```

Handle `NumberFormatException` .

Program 4: Null Name Checker

Create a method:

```
void printNameLength(String name)
```

If name is null, handle `NullPointerException` .

Then improve the same program without exception by checking:

```
if (name != null)
```

Write which approach is better and why.

Program 5: File Reader

Create a program to open file:

```
student.txt
```

Requirements:

- Use `FileReader` .
- Handle `FileNotFoundException` .
- Print meaningful message if file does not exist.

Program 6: File Reader With Finally

Create a program using `FileReader` .

Requirements:

- Declare `FileReader reader = null` .
- Open file inside try.
- Close file inside finally.
- Handle `IOException` while closing.

Program 7: Age Validation Using `throw`

Create class `VoterService` .

Method:

```
void checkAge(int age)
```

Rules:

- If age is less than 18, throw `IllegalArgumentException` .

- Otherwise print "Allowed to vote" .
- Handle exception in main method.

Program 8: Method With throws

Create class `FileService` .

Method:

```
void readFile() throws FileNotFoundException
```

In main method:

- Create object of `FileService` .
- Call `readFile()` .
- Handle exception using try-catch.

Program 9: Custom Unchecked Exception

Create custom exception:

```
ProductNotFoundException extends RuntimeException
```

Create class `ProductService` with method:

```
void findProduct(int productId)
```

Rules:

- If product id is not 101 , throw `ProductNotFoundException` .
- Otherwise print "Product found" .
- Handle exception in main method.

Program 10: Custom Checked Exception

Create custom exception:

```
InsufficientBalanceException extends Exception
```

Create class `BankAccount` with:

- `balance`
- constructor
- `withdraw(double amount) throws InsufficientBalanceException`

Rules:

- If amount is greater than balance, throw custom exception.
- Otherwise deduct balance and print remaining balance.
- Handle exception in main method.

Program 11: Login Validation

Create custom unchecked exception:

```
InvalidLoginException
```

Create class `LoginService` with:

```
void login(String username, String password)
```

Rules:

- Correct username: `"admin"`
- Correct password: `"java123"`
- If username or password is wrong, throw `InvalidLoginException`.
- Handle exception in main method.

Program 12: Backend-Style Product API Simulation

Create:

```
ProductNotFoundException  
InvalidProductIdException  
ProductService  
Main
```

In `ProductService`, create method:

```
void getProductById(int productId)
```

Rules:

- If productId is less than or equal to 0, throw `InvalidProductIdException`.
- If productId is not 101, throw `ProductNotFoundException`.
- If productId is 101, print product details.

In main method:

- Call service method inside try.
- Use multiple catch blocks.
- Print user-friendly messages.

Part F: Return With Try-Catch-Finally

Write output and explanation for each program.

Question 1

```
class Test {
    int show() {
        try {
            return 100;
        } finally {
            System.out.println("Finally");
        }
    }
}
```

Question 2

```
class Test {
    int show() {
        try {
            int result = 10 / 0;
            return 100;
        } catch (ArithmeticException e) {
            return 200;
        } finally {
            System.out.println("Finally");
        }
    }
}
```

Question 3

```
class Test {
    int show() {
        try {
            return 100;
        } catch (Exception e) {
            return 200;
        } finally {
            return 300;
        }
    }
}
```

Question 4

```
class Test {  
    int show() {  
        int number = 10;  
  
        try {  
            return number;  
        } finally {  
            number = 20;  
        }  
    }  
}
```

Question 5

```
class Test {  
    StringBuilder show() {  
        StringBuilder sb = new StringBuilder("Java");  
  
        try {  
            return sb;  
        } finally {  
            sb.append(" Backend");  
        }  
    }  
}
```

Part G: Concept Mapping

Complete the table:

Situation	Concept Used
Risky code is written here	
Exception handling code is written here	
Cleanup code is written here	
Manually throwing exception	
Declaring exception in method signature	
Compiler forces handling	
Compiler does not force handling	
Wrong array index	
Divide by zero	
Invalid String to number conversion	
Null reference method call	
Application-specific exception	
Exception moves from called method to caller method	
Full exception details with line number	

Part H: Difference Questions

Write differences:

1. Error vs Exception
2. Compile-time error vs Runtime exception
3. Checked exception vs Unchecked exception
4. `try` vs `catch`
5. `throw` vs `throws`
6. `final` vs `finally` vs `finalize`
7. `getMessage()` vs `printStackTrace()`
8. Specific catch vs generic catch
9. Custom checked exception vs custom unchecked exception
10. Exception handling vs input validation

Part I: Real-World Thinking Questions

1. In an online payment system, what exceptions can occur?
2. In a login API, why should we not show technical exception details to user?
3. In a banking app, where can we use custom exception?
4. In a file upload feature, which checked exceptions may occur?
5. In a backend service, why should we log exception details?

6. Why is catching exception silently dangerous?
7. Why should business exceptions have meaningful names?
8. Why is `finally` useful for closing resources?

Submission Checklist

Before submitting, check:

- Did you identify checked and unchecked exceptions?
- Did you use specific catch blocks?
- Did you put catch blocks in correct order?
- Did you use `finally` for cleanup examples?
- Did you explain `throw` and `throws` correctly?
- Did you create custom exception classes properly?
- Did you write expected output before running output questions?
- Did you avoid empty catch blocks?
- Did you write meaningful messages for users?

